

PLACEMENT PORTAL APPLICATION

Final Project Report

BY SIDDHARTH NATH TIWARI

24F2005039

|Modern Application Development - II |

May, 2026 Term

1. Student Details

Name: Siddharth Nath Tiwari

Roll Number: 24f2005039

Email: 24f2005039@ds.study.iitm.ac.in

About Me: I am a student in the IIT Madras BS Data Science and Applications program with a strong focus on full-stack web development. I enjoy building robust, production-ready applications utilizing frameworks like Vue.js and Flask, with an interest in optimizing backend performance and designing clean, intuitive user interfaces.

2. Project Details

Project Title: PLACEMENT PORTAL APPLICATION

Problem Statement: To design and build a centralized, web-based platform that bridges the gap between university administrators, recruiting companies, and students. The system must streamline the creation of placement drives, automate application tracking, and provide dedicated dashboards for role-based management and analytics.

Approach: The application features a decoupled architecture using Vue.js for a dynamic, reactive Single Page Application (SPA) frontend and Flask as a robust backend API. It implements secure JWT-based role authentication (Admin, Company, Student). To handle high traffic and complex queries, the system utilizes Redis for query caching and search debouncing, alongside Celery for handling long-running background tasks like CSV data exports.

3. AI/LLM Declaration

I used AI tools to assist in debugging complex Vue.js reactivity issues (such as component state crashes during UI updates), creating templates, and messages that are sent on gmail and webhooks like Gchat.

- The extent of AI/LLM usage is around 15–20%, heavily focused on template designing, structural refactoring, and code review.
 - All core application architecture, business logic, routing, and final integrations were implemented and tested manually.
-

4. Technologies and Frameworks Used

Technology / Library	Purpose
Vue.js (v3)	Frontend framework for building reactive, single-page dashboards
Flask	Core backend REST API framework
SQLAlchemy	Object Relational Mapper (ORM) for database management
Redis	In-memory data store used for API route caching and search optimization
Celery	Asynchronous task queue for handling background CSV report generation
JWT (JSON Web Tokens)	Secure, stateless user authentication and role authorization
Bootstrap 5	Frontend styling, grid layouts, and responsive UI components
SQLite	Lightweight local relational database for data persistence

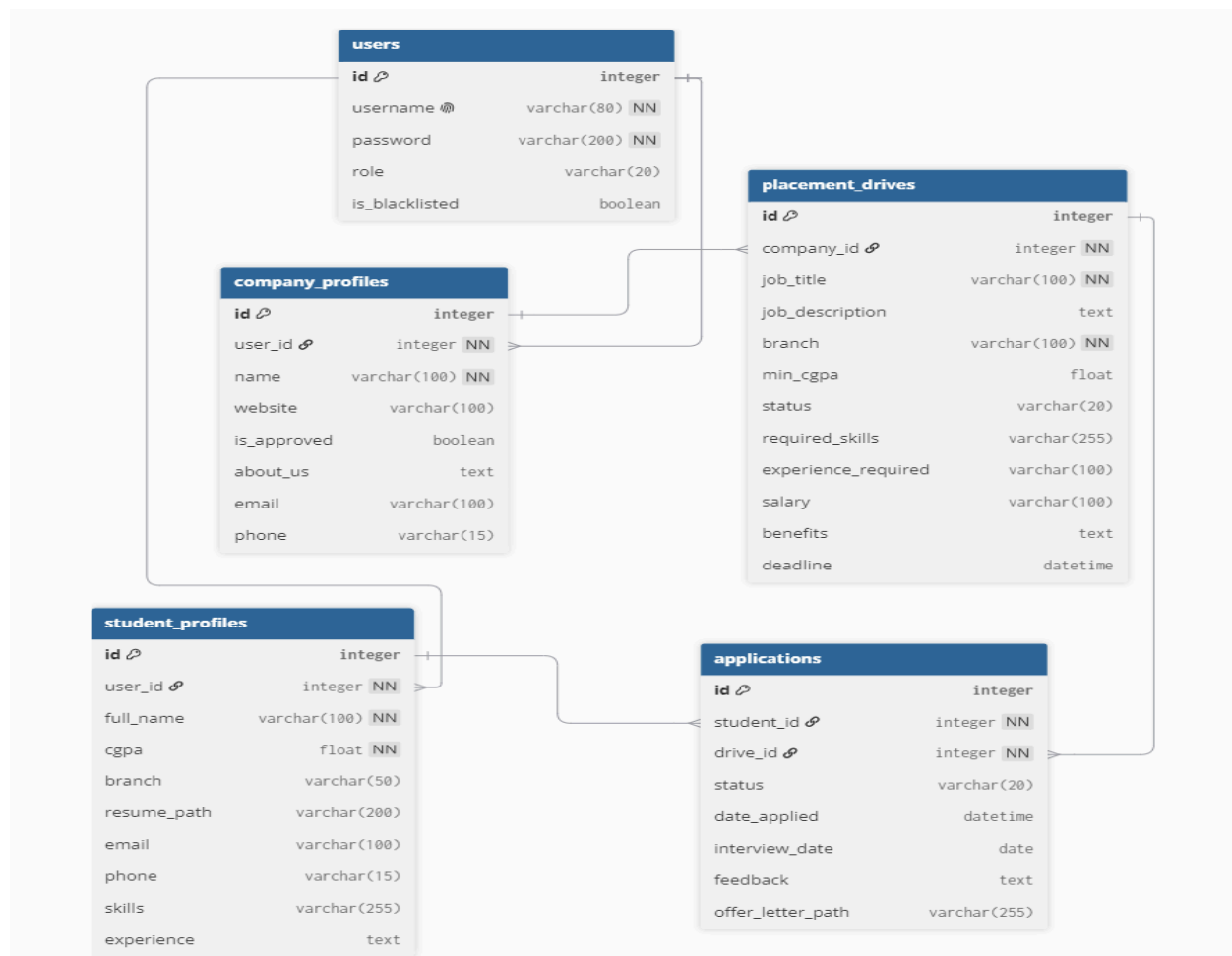
5. Database Schema / ER Diagram

Core Tables:

- **User** — Manages authentication and roles (*id, email, password_hash, role*)
- **StudentProfile** — Stores student details (*id, user_id, full_name, cgpa, branch, resume_path*)
- **CompanyProfile** — Stores employer details (*id, user_id, name, about_us, website, status*)
- **PlacementDrive** — Logs job postings (*id, company_id, job_title, min_cgpa, branch, deadline, status*)
- **Application** — Tracks student applications to drives (*id, student_id, drive_id, status, interview_date*)

Relationships:

- One-to-One → User → StudentProfile / CompanyProfile
- One-to-Many → CompanyProfile → PlacementDrive
- One-to-Many → PlacementDrive → Application
- One-to-Many → StudentProfile → Application



6. API Resource Endpoints

Endpoint	Method	Description
<code>/api/login</code>	POST	Authenticates user credentials and returns JWT access token
<code>/api/admin/search</code>	GET	Redis-cached global search across students, companies, and drives
<code>/api/student/apply/<drive_id></code>	POST	Creates a new student application with deadline validation
<code>/api/admin/company-action/<id></code>	POST	Admin mutation to approve, reject, or blacklist a company
<code>/api/company/export-csv</code>	POST	Triggers asynchronous Celery task to generate applicant history

Included separately in the submission ZIP as `api.yaml`.

7. Architecture and Features (optional)

Architecture Overview:

- Frontend (/frontend): Built with Vue.js, utilizing axios for HTTP requests, vue-router for navigation, and component-based architecture for the Dashboards.
- Backend (/backend): app.py serves as the entry point, routing requests to specific controller functions.
- State & Caching: Redis caches heavy GET requests to reduce SQLite load.
- Background Workers: Celery instances run concurrently to process heavy file-writing operations without blocking the main Flask thread.

Implemented Features:

- Role-based access control (RBAC) with dedicated Dashboards (Admin, Company, Student).
- Real-time global search with 300ms debouncing and multi-field SQL or_ querying.
- Optimistic UI updates for instant frontend feedback on database mutations.
- Application pipeline management (Applied → Shortlisted → Selected → Rejected).
- Background asynchronous CSV generation with a polling mechanism for downloads.
- File handling for PDF resume and offer letter uploads/downloads.

8. Video Presentation

Drive Link:

 MAD2_Project_DEMO.mp4

<https://drive.google.com/file/d/1xc8NzVXTLd3IJjDolmtTMJno87CihEmC/view?usp=sharing>
